Gantt Chart Visualization Documentation

Real-time Process Execution Timeline with PyQtGraph

Updated: 2026-03-07

Warning: This documentation was generated by an AI assistant. However I have reviewed this file and made changes accordingly if needed.

Abstract

The Gantt Chart module provides a comprehensive visualization of transformer training processes over time. It displays process execution timelines with task-based row grouping, epoch-colored bars, and synchronized crosshair interaction with resource monitoring plots. The implementation supports both manual and automatic refresh, epoch range filtering, and intelligent handling of parallel vs. sequential task execution.

Contents

1 Overview

The Gantt Chart component is integrated into the `Training_performance_plotter.py` application as a toggleable panel in the Performance Reports tab. It visualizes process execution data from `eval_results/processes.json` with the following key features:

- **Task-Based Row Grouping**: Same tasks across different epochs share rows; parallel tasks get separate rows
- **Epoch-Colored Bars**: Different colors for different epochs with legend display
- **Time Synchronization**: Uses the same time reference as resource plots (earliest process init time)
- **Crosshair Sync**: Red vertical line on hover synchronizes with CPU/GPU/RAM plots
- **Epoch Range Filtering**: Respects selected epoch range from control panel
- **Refresh Functionality**: Manual refresh button and optional 5-second auto-refresh
- **Scrollable Labels**: Fixed label panel on the left that scrolls with the chart

2 UI Layout

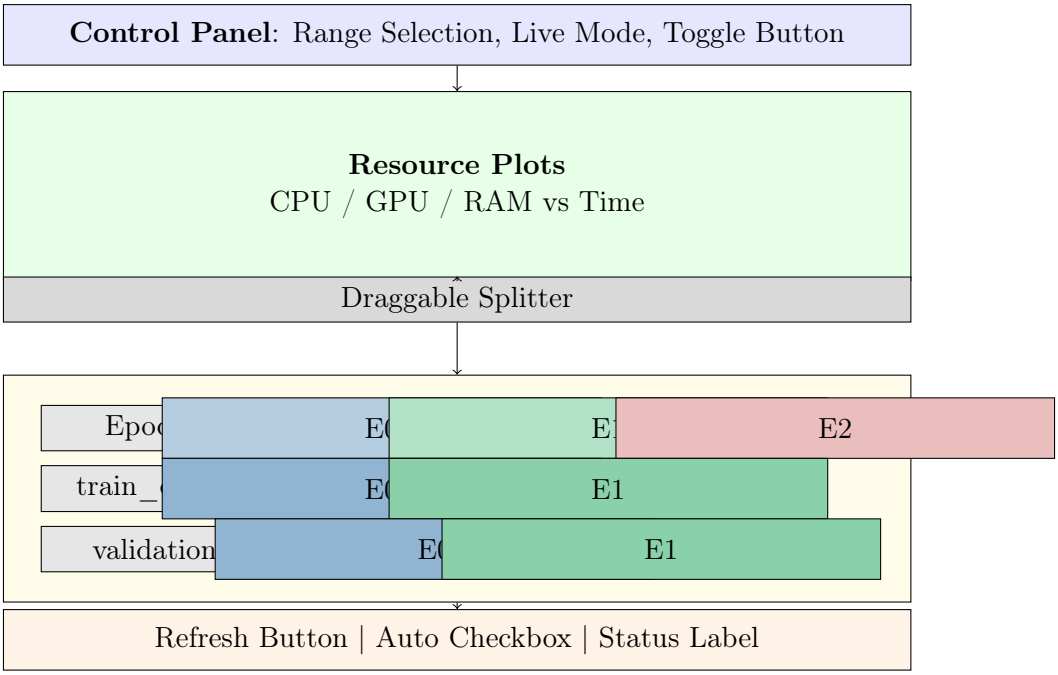
The Gantt Chart panel uses a vertical splitter layout below the resource plots:

```
# Main container with controls
self.gantt_container = QtWidgets.QWidget()

# Top: Epoch legend
self.gantt_legend_layout = QtWidgets.QHBoxLayout()

# Middle: Scroll area with labels and chart
self.gantt_scroll_area = QtWidgets.QScrollArea()
self.gantt_content_widget = QtWidgets.QWidget()

# Bottom: Refresh controls
self.gantt_refresh_btn = QtWidgets.QPushButton("Refresh")
self.gantt_auto_refresh_checkbox = QtWidgets.QCheckBox("Auto")
self.gantt_status_label = QtWidgets.QLabel()
```

Listing 1: Gantt Chart Container Layout

2.1 Visual Layout Diagram

Note: I have reviewed this section of the pdf, and I couldn't fix the diagram, but I am going to leave it as it is legible.



3 Class Reference

3.1 GanttChartWidget

The `GanttChartWidget` class extends `pg.GraphicsLayoutWidget` to provide the Gantt chart visualization.

3.1.1 Constructor

```
class GanttChartWidget(pg.GraphicsLayoutWidget):
    """
    A Gantt chart widget for visualizing process execution timelines.

    Features:
    - Task-based row grouping (same task across epochs shares row)
    - Epoch-colored bars with labels
    - Crosshair synchronization with resource plots
    - Epoch range filtering support
    """

    def __init__(self, parent=None):
        super().__init__(parent)
        self.plot_item = self.addPlot(row=0, col=0)
        self.plot_item.setMenuEnabled(False)
        self.plot_item.setMouseEnabled(x=True, y=False)
        self.plot_item.hideButtons()
        self.crosshair_line = None
        self.processes_data = []
        self.base_time = 0
```

3.1.2 Key Attributes

3.2 Task Grouping and Row Assignment

The Gantt Chart uses intelligent row assignment to handle both sequential and parallel processes:

Attribute	Type	Description
plot_item	PlotItem	Main pyqtgraph plot for rendering bars
crosshair_line	InfiniteLine	Red vertical line for time synchronization
processes_data	list	Filtered process data for display
base_time	float	Reference time (earliest process init)
row_assignments	dict	Maps task names to row indices
epoch_colors	dict	Maps epoch numbers to color values

3.2.1 Base Task Name Extraction

```python
def get_base_task_name(process_name: str) -> str:
    """
    Extract base task name by removing numeric suffixes.

    Examples:
    - 'train_epoch_05' -> 'train_epoch'
    - 'EncoderBlock_L0' -> 'EncoderBlock'
    - 'E5_B0042_SelfAttention' -> 'E5_B0042_SelfAttention'
    """
    # Remove trailing digits and underscores
    base_name = re.sub(r'_\d+$', '', process_name)
    return base_name or process_name
```

3.2.2 Time-Aware Row Assignment

```python
def assign_to_rows(processes: list) -> tuple:
    """
    Assign processes to rows based on task name and time overlap.

    Logic:
    1. Sort processes by start time
    2. For each process, find first row where it doesn't overlap
    3. Create new row if no suitable row exists

    Returns:
        (row_labels, row_processes): Labels for y-axis and
                                     processes per row
    """
    # Sort by start time
    sorted_processes = sorted(processes,
                              key=lambda p: p.get('start', 0))

    rows = []  # Each row is a list of (start, end, process)

    for proc in sorted_processes:
        base_name = get_base_task_name(proc['name'])
        start = proc.get('start', 0)
        end = proc.get('end', start)

        assigned = False
        for i, row in enumerate(rows):
            # Check for time overlap
            overlap = any(s < end and start < e
                          for s, e, _ in row)
```

```
            if not overlap:
                rows[i].append((start, end, proc))
                assigned = True
                break

        if not assigned:
            rows.append([(start, end, proc)])

    # Generate labels with suffixes for parallel rows
    row_labels = []
    task_row_counts = {}
    for row in rows:
        if row:
            base_name = get_base_task_name(row[0][2]['name'])
            task_row_counts[base_name] = \
                task_row_counts.get(base_name, 0) + 1

    # ... create labels with (2), (3) suffixes for parallel rows
```

Row Assignment Examples:

Processes	Result	Reason
Sequential train_epoch_00, _01, _02	1 row	No time overlap
Parallel EncoderBlocks	Multiple rows	Time conflict
LayerNorm in different epochs	1 row	Sequential execution

3.3 Data Loading

3.3.1 Loading Process Data

```
def load_process_data(self, epoch_range=None):
    """
    Load process data from processes.json.

    Args:
        epoch_range: Optional tuple (start_epoch, end_epoch)
                     to filter processes
    """
    processes_file = Path("eval_results/processes.json")

    if not processes_file.exists():
        return

    with open(processes_file, 'r') as f:
        data = json.load(f)

    all_processes = []
    for uid, proc_data in data.get('processes', {}).items():
        epoch = Process.get_epoch_from_uid(uid)

        # Apply epoch range filter
        if epoch_range and epoch_range[0] is not None:
            if epoch < epoch_range[0] or epoch > epoch_range[1]:
                continue

        timeline = proc_data.get('timeline', {})
```

```
        init_time = timeline.get('initialized', 0)
        term_time = timeline.get('terminated', -1)

        if term_time < 0:
            term_time = init_time  # Ongoing process

        all_processes.append({
            'name': proc_data.get('name', 'Unknown'),
            'uid': uid,
            'epoch': epoch,
            'start': init_time,
            'end': term_time,
            'duration': term_time - init_time
        })

    self.processes_data = all_processes
    return self.processes_data
```

3.4 Rendering

3.4.1 Rendering Gantt Bars

```
def render_gantt(self):
    """
    Render the Gantt chart with colored bars.

    Each bar represents a process execution:
    - X position: Start time (relative to base_time)
    - Width: Duration
    - Y position: Assigned row
    - Color: Based on epoch number
    - Label: Epoch identifier (E0, E1, etc.)
    """
    self.plot_item.clear()

    if not self.processes_data:
        return

    # Calculate base time from earliest process
    self.base_time = min(p['start'] for p in self.processes_data)

    # Assign rows using time-aware grouping
    row_labels, row_processes = assign_to_rows(self.processes_data)

    # Generate epoch colors
    unique_epochs = sorted(set(p['epoch']
                               for p in self.processes_data))
    self.epoch_colors = {}
    for i, epoch in enumerate(unique_epochs):
        self.epoch_colors[epoch] = pg.intColor(i,
                                   len(unique_epochs))

    # Draw bars
    for row_idx, row in enumerate(row_processes):
        for start, end, proc in row:
            x = start - self.base_time
            width = end - start
```

```
            y = row_idx

            epoch = proc['epoch']
            color = self.epoch_colors[epoch]

            # Create bar item
            bar = pg.BarGraphItem(
                x=[x + width/2],
                y=[y],
                width=width,
                height=0.6,
                brush=color,
                pen=pg.mkPen(color='k', width=1)
            )
            self.plot_item.addItem(bar)

            # Add epoch label
            text = pg.TextItem(f"E{epoch}", anchor=(0.5, 0.5))
            text.setPos(x + width/2, y)
            self.plot_item.addItem(text)

    # Set Y axis labels
    self.plot_item.getAxis('left').setTicks(
        [(i, label) for i, label in enumerate(row_labels)]
    )
```

4 Crosshair Synchronization

The Gantt Chart crosshair synchronizes with the resource plots' vertical line:

```
def set_crosshair_position(self, x_pos: float):
    """
    Set the crosshair line position.

    Called when hovering over resource plots to show
    corresponding time position in Gantt chart.

    Args:
        x_pos: X coordinate in seconds (relative to base_time)
    """
    if self.crosshair_line is None:
        self.crosshair_line = pg.InfiniteLine(
            angle=90,
            movable=False,
            pen=pg.mkPen(color='r', width=2, style=Qt.PenStyle.
   SolidLine)
        )
        self.plot_item.addItem(self.crosshair_line)

    self.crosshair_line.setValue(x_pos)
```

4.1 Event Handling in Resource Plots

```
def _on_pReport_plot_hover(self, event):
    """
    Handle mouse hover on resource plots.
```

```
    Updates crosshair position in Gantt chart when hovering
    over CPU/GPU/RAM plots.
    """
    if not self.gantt_chart_visible:
        return

    # Get mouse position in plot coordinates
    mouse_point = self.cpu_plot.vb.mapSceneToView(event.scenePos())
    time_x = mouse_point.x()

    # Update Gantt chart crosshair
    self.gantt_chart.set_crosshair_position(time_x)
```

5 Refresh Functionality

5.1 Manual Refresh

```
def _refresh_gantt_chart(self):
    """
    Manually refresh Gantt chart data.

    Reloads process data from JSON and re-renders the chart.
    Updates timestamp label to show last refresh time.
    """
    self._load_and_render_gantt()

    # Update status label
    current_time = datetime.now().strftime("%H:%M:%S")
    self.gantt_status_label.setText(f"Last updated: {current_time}")
```

5.2 Auto-Refresh Timer

```
def _on_gantt_auto_refresh_changed(self, state):
    """
    Toggle auto-refresh timer.

    When enabled, refreshes the Gantt chart every 5 seconds.
    Useful for monitoring live training.

    Args:
        state: Qt.CheckState (Checked or Unchecked)
    """
    if state == Qt.CheckState.Checked:
        self.gantt_refresh_timer.start(5000)  # 5 seconds
    else:
        self.gantt_refresh_timer.stop()
```

6 Time Synchronization

To ensure the Gantt Chart aligns correctly with resource plots, both use the same time reference:

```
def _get_process_min_time(self) -> float:
    """
```

```
    Get the earliest process initialization time from processes.json.

    This time serves as t=0 for both Gantt chart and resource plots,
    ensuring visual alignment between both views.

    Returns:
        Unix timestamp of earliest process, or None if no processes
    """
    processes_file = Path("eval_results/processes.json")

    if not processes_file.exists():
        return None

    with open(processes_file, 'r') as f:
        data = json.load(f)

    min_time = float('inf')
    for proc_data in data.get('processes', {}).values():
        timeline = proc_data.get('timeline', {})
        init_time = timeline.get('initialized', 0)
        if init_time > 0 and init_time < min_time:
            min_time = init_time

    return min_time if min_time != float('inf') else None
```

6.1 Resource Plot Time Normalization

```
def update_pReport_plots(self):
    """
    Update resource plots with synchronized time reference.

    Uses process min time as t=0 to align with Gantt chart.
    """
    df = pd.read_csv(self.file_sec)

    # Normalize time using process min time
    process_min_time = self._get_process_min_time()
    if process_min_time is not None:
        df["time"] -= process_min_time
    else:
        df["time"] -= df["time"].iloc[0]  # Fallback

    # ... rest of plotting code
```

7 Epoch Legend

The epoch legend shows color mappings for each visible epoch:

```
def _update_gantt_legend(self):
    """
    Update the epoch color legend above the Gantt chart.

    Creates colored checkboxes showing each epoch's color.
    Only shows epochs currently visible in the filtered range.
    """
    # Clear existing legend items
```

```
    while self.gantt_legend_layout.count():
        item = self.gantt_legend_layout.takeAt(0)
        if item.widget():
            item.widget().deleteLater()

    # Add label
    self.gantt_legend_layout.addWidget(
        QtWidgets.QLabel("Epochs:")
    )

    # Add epoch color boxes
    if hasattr(self.gantt_chart, 'epoch_colors'):
        for epoch, color in sorted(
            self.gantt_chart.epoch_colors.items()
        ):
            # Create colored checkbox
            checkbox = QtWidgets.QCheckBox(f"E{epoch}")
            checkbox.setChecked(True)

            # Set checkbox background color
            color_hex = color.name()
            checkbox.setStyleSheet(f"""
                QCheckBox {{
                    background-color: {color_hex};
                    padding: 2px 8px;
                    border-radius: 3px;
                }}
            """)

            self.gantt_legend_layout.addWidget(checkbox)

    # Add stretch to push everything left
    self.gantt_legend_layout.addStretch()
```

8 Integration with Main Application

8.1 Toggle Visibility

```
def _toggle_gantt_chart(self):
    """
    Toggle Gantt chart visibility.

    Shows/hides the Gantt chart panel and adjusts the UI layout.
    Loads process data when first shown.
    """
    self.gantt_chart_visible = not self.gantt_chart_visible

    if self.gantt_chart_visible:
        self.gantt_container.show()
        self.gantt_toggle_btn.setText("Hide Gantt Chart")

        # Load data if not already loaded
        if not self.gantt_chart.processes_data:
            self._refresh_gantt_chart()
    else:
        self.gantt_container.hide()
        self.gantt_toggle_btn.setText("Show Gantt Chart")
```

8.2 Splitter Configuration

```
# Create splitter between plots and Gantt
self.main_splitter = QtWidgets.QSplitter(
    Qt.Orientation.Vertical
)

# Add plots container (top)
self.main_splitter.addWidget(self.plots_container)

# Add Gantt container (bottom)
self.main_splitter.addWidget(self.gantt_container)

# Set initial sizes (70% plots, 30% Gantt)
self.main_splitter.setSizes([700, 300])

# Make both sections stretchable
self.main_splitter.setStretchFactor(0, 1)
self.main_splitter.setStretchFactor(1, 1)
```

9 Usage Examples

9.1 Basic Usage

1. Start the Performance Plotter: `python Training_performance_plotter.py`
2. Go to "Performance Reports" tab
3. Click "Show Gantt Chart" button
4. The Gantt chart loads process data from `eval_results/processes.json`

9.2 Viewing Specific Epoch Range

1. Enter start and end epochs in the range fields
2. Click "Apply Range"
3. Gantt chart updates to show only processes from selected epochs

9.3 Live Training Monitoring

1. Enable "LIVE MODE" checkbox
2. Enable "Auto" checkbox in Gantt chart panel
3. Gantt chart refreshes every 5 seconds during training
4. New processes appear automatically as they complete

9.4 Synchronized Crosshair

1. Hover over CPU, GPU, or RAM plot
2. Red vertical line appears at that time position
3. Same line appears in Gantt chart at corresponding time
4. See which processes were executing at that moment

10 File Locations

File	Description
`eval_results/processes.json`	Process execution data with timestamps
`Training_performance_plotter.py`	Main application with Gantt chart integration

11 Notes and Limitations

- **Data Source:** Requires `processes.json` to be generated by training script
- **Time Accuracy:** Uses process init/termination times; may not capture sub-process timing
- **Row Limit:** Large numbers of parallel tasks may require vertical scrolling
- **Auto-Refresh:** 5-second interval may cause flicker; use manual refresh for stable view
- **Memory Usage:** Large process files may slow down rendering; epoch filtering recommended
- **Crosshair Sync:** Only active when Gantt chart is visible